Configuration and Booting

Objectives

After completing this lab, you will be able to:

- Create a bootable system capable of booting from the SD card
- Create a bootable system capable of booting from the QSPI flash
- Load the bitstream stored on the SD card or in the QSPI flash memory
- Configure the PL section using the stored bitstream through the PCAP resource
- Execute the corresponding application

Steps

Create a Vivado Project

Launch Vivado and create an empty project, called lab5, using the Verilog language.

1. Open Vivado and click **Create New Project** and click **Next**.
2. Click the Browse button of the *Project Location* field of the **New Project** form, browse to { **labs}** , and click **Select**.
3. Enter **lab5** in the *Project Name* field. Make sure that the *Create Project Subdirectory* box is checked. Click **Next**.
4. Select the **RTL Project** option in the *Project Type* form, and click **Next**.
5. Select **Verilog** as the *Target Language* in the *Add Sources* form, and click **Next**.
6. Click **Next** two times.
7. Select **www.digilentinc.com** for the *PYNQ-Z1* board, **tul.com.tw** for the *PYNQ-Z2* board in the Vendor field, select *PYNQ-Z1__or pynq-z2*, and click **Next**.
8. Click **Finish** to create an empty Vivado project.

Creating the Hardware System Using IP Integrator

Create a block design in the Vivado project using IP Integrator to generate the ARM Cortex-A9 processor based hardware system.

1. In the Flow Navigator, click **Create Block Design** under IP Integrator.
2. Name the block **system** and click **OK**.
3. Click the + button.
4. Once the IP Catalog is open, type zy into the Search bar, and double click on **ZYNQ7 Processing System** entry to add it to the design.
5. Click on *Run Block Automation* in the message at the top of the *Diagram* panel. Leave the default option of *Apply Board Preset* checked, and click **OK**.
6. Double click on the Zynq block to open the *Customization* window.

A block diagram of the Zynq should now be open, showing various configurable blocks of the Processing System.

Configure the I/O Peripherals block to only have QSPI, UART 0, and SD 0 support.

1. Click on the *MIO Configuration* panel to open its configuration form.
2. Expand the *IO Peripherals* on the right.
3. Uncheck *ENET 0*, *USB 0*, and *GPIO* > *GPIO MIO,* leaving *UART 0* and *SD 0* selected.
4. Click **OK**.

   The configuration form will close and the block diagram will be updated.
5. Using wiring tool, connect FCLK_CLK0 to M_AXI_GP0_ACLK.
6. Select the *Diagram* tab, and click on the (Validate Design) button to make sure that there are no errors.

Export the Design to the SDK and create the software projects

Create the top-level HDL of the embedded system, and generate the bitstream.

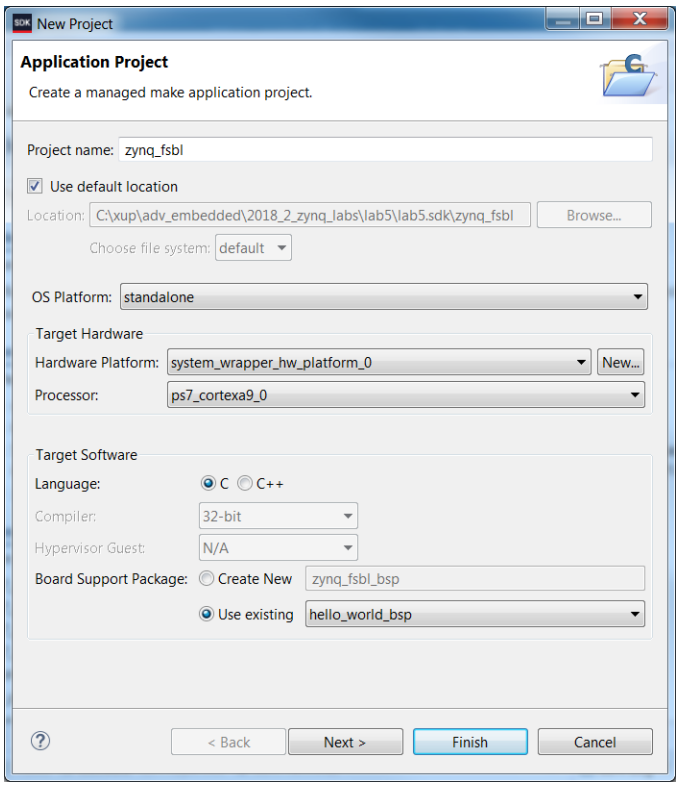
1. In Vivado, select the *Sources tab*, expand the *Design Sources,* right-click the *system.bd* and select **Create HDL Wrapper** and click **OK.**
2. Click on **Generate Bitstream** and click **Generate**. Click **Save** to save the project, and **Yes** if prompted to run the processes. Click **OK** to launch the runs.
3. When the bitstream generation process has completed successfully, click **Cancel**.

Export the design to the SDK and create the Hello World application.

1. Export the hardware configuration by clicking **File > Export > Export Hardware...**
2. Click the box to *Include Bitstream*, then click **OK**
3. Launch SDK by clicking **File > Launch SDK** and click **OK**
4. In SDK, select **File** > **New** > **Application Project.**
5. Enter **hello_world** in the project name field, and leave all other settings as default.
6. Click **Next** and make sure that the *Hello World* application template is selected, and click **Finish** to generate the application.
7. Right click on hello_world_bsp and click **Board Support Package Settings**
8. Tick to include *xilffs* click **OK** (This is required for the next step to create the FSBL).

Create a first stage bootloader (FSBL).

1. Select **File** > **New > Application Project.**
2. Enter **zynq_fsbl** as the project name, select the *Use existing* standalone Board Support Package option with **hello_world_bsp** , and click **Next**.



Creating the FSBL Application

3. Select *Zynq FSBL* in the **Available Templates** pane and click **Finish**.

   A zynq_fsbl project will be created which will be used in creating the BOOT.bin file. The BOOT.bin file will be stored on the SD card which will be used to boot the board.

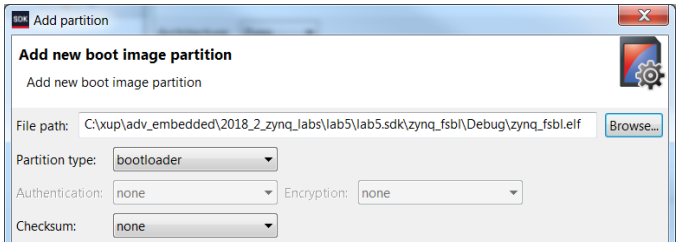
Create the Boot Images and Test

Using the Windows Explorer, create a directory called *image* under the lab5 directory. You will create the BOOT.bin file using the FSBL, system_wrapper.bit and hello_world.elf files.

1. Using the Windows Explorer, create a directory under the *lab5* directory and call it **image**.
2. In the SDK, select **Xilinx > Create Boot Image**.

   Click on the Browse button of the **Output BIF** field, browse to **{labs}\lab5\image** and click **Save** (leaving the default name of output.bif)
3. Click on the **Add** button of the *Boot image partitions,* click the Browse button in the Add Partition form, browse to **{labs}\lab5\lab5.sdk\zynq_fsbl\Debug** directory (this is where the FSBL was created), select *zynq_fsbl.elf* and click **Open**.

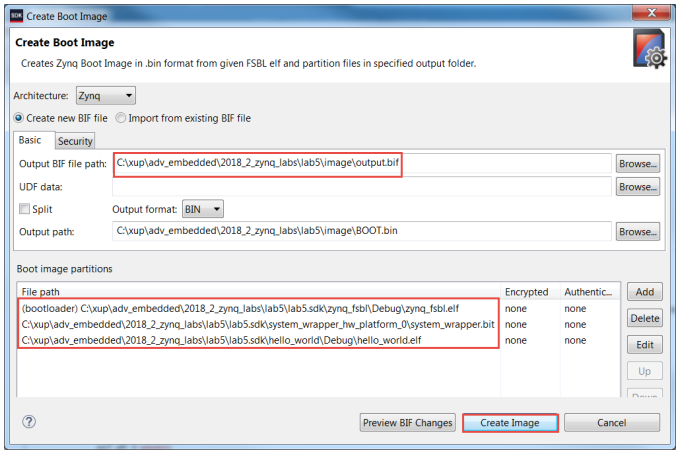
   Note the partition type is bootloader, then click **OK.**



Adding FSBL partition

4. Click on the **Add** button of the *Boot image partitions* and add the bitstream, **system_wrapper.bit** , from **{labs}\lab5\lab5.sdk\system_wrapper_hw_platform_0** and click **OK**.
5. Click on the **Add** button of the *Boot image partitions* and add the software application, **hello_world.elf** , from **{labs}\lab5\lab5.sdk\hello_world\Debug** and click **OK**.
6. Click the **Create Image** button.

   The BOOT.bin and the output.bif files will be created in the **{labs}\lab5\image** directory. We will use the BOOT.bin for the SD card boot up.



Creating BOOT.bin image file

7. Insert a blank SD/MicroSD card (FAT32 formatted) in a Card reader, and using the Windows Explorer, copy the BOOT.bin file from the **image** folder in to the SD/MicroSD card.

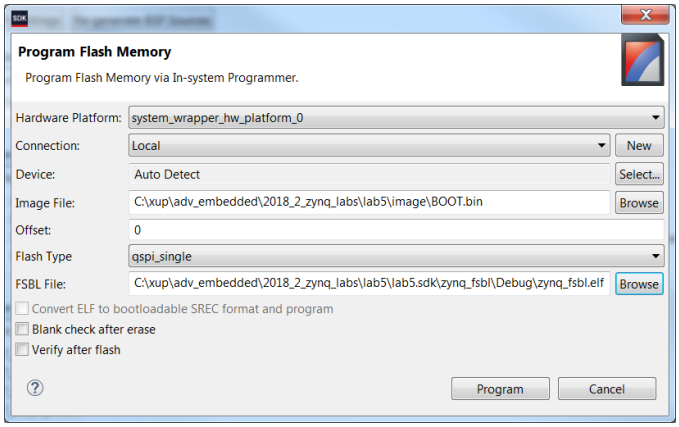
Set the board in SD card boot mode. Test the functionality by starting a Terminal emulator program and powering ON the board.

1. Set the board in the SD card boot mode.
2. Insert the SD/MicroSD card into the board.
3. Power ON the board.
4. Connect your PC to the UART port with the provided micro-USB cable, and start Terminal or SDK Terminal in SDK or a Terminal emulator program setting it to the current COM port and 115200 baudrate.
5. You should see the **Hello World** message in the terminal emulator window. If you don't see it, then press the PS_RST/PS_SRST button on the board.
6. Once satisfied power OFF the board and remove the SD card.

Set the board in the QSPI boot mode. Power ON the board, Program the QSPI using the Flash Writer utility, and test by pressing PS-RST button.

1. Set the board in the QSPI mode.
2. Power ON the board.
3. Connect your PC to the UART port with the provided micro-USB cable, and start a Terminal emulator program setting it to the current COM port and an 115200 baud rate.
4. Select **Xilinx > Program Flash**.
5. In the *Program Flash Memory* form, click the **Browse** button of the Image File field, browse to the **{labs}\lab5\image** directory, select **BOOT.bin** file, and click **Open**.
6. Click on the FSBL File Browse button, browse to **{labs}\lab5\lab5.sdk\zynq_fsbl\Debug** directory (this is where the FSBL was created), select *zynq_fsbl.elf* and click **Open**.
7. In the *Offset* field enter **0** as the offset and click the **Program** button.

   The QSPI flash will be programmed.



Program Flash Memory form

8. Disconnect and reconnect the Terminal window.
9. Press the PS-RST to see the **Hello World** message in the terminal emulator window.
10. Once satisfied, power OFF the board.

Prepare for the Multi-Applications Boot Using SD Card

The lab1 and lab3 executable files are required in the .bin format before copying to the SD card. The area in memory allocated for each application need to be modified so that they do not overlap each other, or with the main application. The prepared bin files provided in the directory: {sources}\lab5[pynqz1 or pynqz2]\SDCard can be used for copying to the SD card. Follow steps in Appendix A-1 and A-2 if you want to generate by yourself.

Create the lab5_sd application using the provided lab5_sd.c and devcfg.c, devcgf.h, load_elf.s files.

1. Select **File** > **New** > **Application Project.**
2. Enter **lab5_sd** as the project name, click the *Use existing* option in the *Board Support Package* (BSP) field and select *hello_world_bsp*, and then click **Next.**
3. Select *Empty Application* in the **Available Templates** pane and click **Finish**.
4. Select **lab5_sd > src** in the project view, right-click, and select **Import.**
5. Expand the General folder and double-click on **File system,** and browse to the **{sources}\lab5** directory.
6. Select **devcfg.c, devcfg.h, load_elf.s** , and **lab5_sd.c,** and click **Finish.**
7. Change, if necessary, LAB1_ELFBINFILE_LEN, LAB3_ELFBINFILE_LEN, LAB1_ELF_EXEC_ADDR, LAB3_ELF_EXEC_ADDRvalues and save the file.

Create the SD Card Image and Test

Using the Windows Explorer, create the SD_image directory under the lab5 directory. You will first need to create the bin files from lab1 and lab3.

1. Using the Windows Explorer, create directory called **SD_image** under the **lab5** directory.
2. In Windows Explorer, copy the **system_wrapper.bit** of the lab1 project into the *SD_image* directory and rename it *lab1.bit*, and do similar for lab3

   {labs}/lab1/lab1.runs/impl_1/system_wrapper.bit -> SD_image /lab1.bit

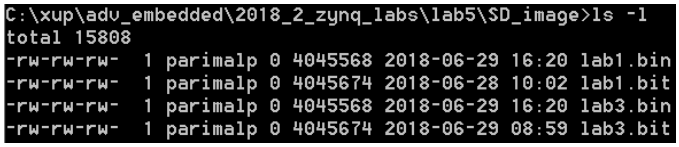
   {labs}/lab3/lab3.runs/impl_1/system_wrapper.bit -> SD_image /lab3.bit

   The XSDK *bootgen* command will be used to convert the bit files into the required binary format. bootgen requires a .bif file which has been provided in the sources/lab5 directory. The .bif file specifies the target .bit files.
3. Open a command prompt by selecting **Xilinx > Launch Shell.**
4. In the command prompt window, change the directory to the bitstreams directory.

   cd {labs}/lab5/SD_image
5. Generate the partial bitstream files in the BIN format using the provided ".bif" file located in the *sources* directory. Use the following command:

   bootgen -image ..\ ..\ ..\2018_2_zynq_sources\lab5\bit_files.bif -w -process_bitstream bin
6. Rename the files *lab1.bit.bin* and *lab3.bit.bin* to *lab1.bin* and *lab3.bin*

7. The size of the file needs to match the size specified in the **lab5_sd.c** file. The size can be determined by checking the file's properties. If the sizes do not match, then make the necessary change to the source code and save it (The values are defined as LAB1_BITFILE_LEN and LAB3_BITFILE_LEN).



Checking the size of the generate bin file

Note that the lab1.bin and lab3.bin files should be the same size.

You will create the BOOT.bin file using the first stage bootloader, system_wrapper.bit and lab5_sd.elf files.

1. In the SDK, select **Xilinx > Create Boot Image**.
2. For the Output BIF file path, click on the Browse button and browse to **{labs}\lab5\SD_image** directory and click **Save**.
3. Click on the **Add** button and browse to **{labs}\lab5\lab5.sdk\zynq_fsbl\debug** , select **zynq_fsbl.elf** , click **Open** , and click **OK**.
4. Click on the **Add** button of the *boot image partitions* field and add the bitstream file, **system_wrapper.bit** , from **{labs}\lab5\lab5.sdk\system_wrapper_hw_platform_0** and click **OK**
5. Click on the **Add** button of the *boot image partitions* field and add the software application, **lab5_sd.elf** , from the **{labs}\lab5\lab5.sdk\lab5_sd\Debug** directory and click **OK**
6. Click the **Create Image** button.

   The BOOT.bin file will be created in the **lab5\SD_image** directory.

Either copy the labxelf.bin files (two) from the sources directory or from the individual directories (if you did the optional part in the previous step and place them in the SD_image directory. Copy all the bin files to the SD card. Configure the board to boot from SD card. Power ON the board. Test the design functionality

1. In Windows explorer, copy the **lab1elf.bin** and **lab3elf.bin** files either from the **{sources}\lab5\ [pynqz1 or pynz2]\SDCard** directory or from the individual directories (if you did the optional parts in the previous step) and place them in the **SD_image** directory.

   {labs}\lab1\lab1.sdk\lab1\Debug\lab1elf.bin -> SD_image

   {labs}\lab3\lab3.sdk\lab3\Debug\lab3elf.bin -> SD_image

1. Insert a blank SD/MicroSD card (FAT32 formatted) in an SD Card reader, and using the Windows Explorer, copy the two bin files, the two elfbin files, and BOOT.bin from the **SD_image** folder in to the SD card.

2. Place the SD/MicroSD card in the board, and set the mode pins to boot the board from the SD card. Connect your PC to the UART port with the provided micro-USB cable.
3. Power ON the board.
4. Start the terminal emulator program and follow the menu. Press the PS_RST/PS_SRST button if you don't see the menu.
5. When finished testing one application, either power cycle the board and verify the second application's functionality, or press the PS_RST/PS_SRST button on the board to display the menu again.
6. When done, power OFF the board.

Create the QSPI application and image

The prepared bin files provided in the directory: {sources}\lab5[pynqz1 or pynqz2]\QSPI can be used for creating the MCS. Appendix B-1 and B-2 lists the steps of how to create such BIN files.

Create the lab5_qspi application using the provided lab5_qspi.c file.

1. Select **File** > **New** > **Application Project.**
2. Enter **lab5_qspi** as the project name, select the *Use existing* option for the Board Support Package, and using the drop-down button select *hello_world_bsp*, and click **Next**.
3. Select *Empty Application* in the **Available Templates** pane and click **Finish**.
4. Select **lab5_qspi>src** in the project view, right-click, and select **Import.**
5. Expand General folder and double-click on **File system,** and browse to the **{sources}\lab5** directory.
6. Select **lab5_qspi.c** and click **Finish.**

   The program should compile successfully and generate the lab5_qspi.elf file.

Using the Windows Explorer, create the QSPI_image directory under the lab5 directory. Create the lab5.mcs file using the zynq_fsbl.elf, system_wrapper.bit, and lab5_qspi.elf files (from the lab5 project), lab1.bin (from the lab1 project) and lab3.bin (from the lab3 project).

1. Using the Windows Explorer, create the **QSPI_image** directory under the **lab5** directory.
2. In the SDK, select **Xilinx > Create Boot Image**.
3. Select **Create new BIF file** option, click the *Browse* button of the BIF file path field and browse to *{labs}\lab5\QSPI_image* directory, select *output.bif*, and click **Save**.
4. Click on the **Add** button of the *Boot image partitions* window.
5. Click on the *Browse* button of the *File Path* field, browse to *{labs}\lab5\lab5.sdk\zynq_fsbl\Debug, select zynq_fsbl.elf*, click **Open** , and then click **OK**.
6. Click on the **Add** button of the *boot image partitions* field and add the bitstream file, **system_wrapper.bit** , from **{labs}\lab5\lab5.sdk\system_wrapper_hw_platform_0** and click **OK**

7. Click on the **Add** button again of the *Boot image partition* field again and add the **lab1.bin** , either of created boot image of the lab1 project (in *{labs}\lab1)* or from the provided *{sources}\lab5(pynqz1or pynqz2)\QSPI* directory. Click **Open**.
8. Enter **0x400000** in the *Offset* field and click **OK**.
9. Similarly, click on the **Add** button again of the *Boot image partition* field again and add the **lab3.bin** , either of the created boot image of the lab3 project (in *{labs}\lab3)* or from the provided *{sources}\lab5(pynqz1 or pynqz2)\QSPI* directory. Click **Open**.
10. Enter **0x800000** in the *Offset* field and click **OK**.
11. Make the window bigger (taller), if necessary, so that you can see the **Output**** path** field,
12. Change the output filename to **lab5.mcs** and the location to *{labs}\lab5\QSPI_image (if necessary).*
13. Click the **Create Image** button.

    The lab5.mcs file will be created in the **lab5\QSPI_image** directory.

Set the board in the QSPI boot mode. Power ON the board. Program the QSPI using the Flash Writer utility.

1. Set the board in the QSPI mode. Power ON the board.
2. Select **Xilinx > Program Flash**.
3. In the *Program Flash Memory* form, click the **Browse** button, and browse to the **{labs}\lab5\QSPI_image** directory, select **lab5.mcs** file, and click **Open**.
   A solution mcs file is provided in the {sources}\lab5{board}\QSPI directory as lab5.zip, unzip it and use it if you have skipped the previous step of generate QSPI image.
4. In the *Offset* field enter **0** as the offset and click the **Program** button.

   The QSPI flash will be programmed. It may take up to 4 minutes.

Test the QSPI Multi-Applications

Connect to the serial port. Press the PS-SRST button and test the functionality.

1. Start the terminal emulator session and press PS-SRST button to see the menu.
2. Follow the menu and test the functionality of each lab.

   Press 1 to load and execute lab1 or 2 to load and execute lab3. After each lab is executed, the lab5 gets loaded displaying the menu. Note that lab1 execution terminates when all slide switches are ON (i.e. 0xF) and lab3 execution terminates after it counts from 0 to 15.
3. Once satisfied, power OFF the board.
4. Close SDK and Vivado programs by selecting **File > Exit** in each program.
5. Turn OFF the power on the board.

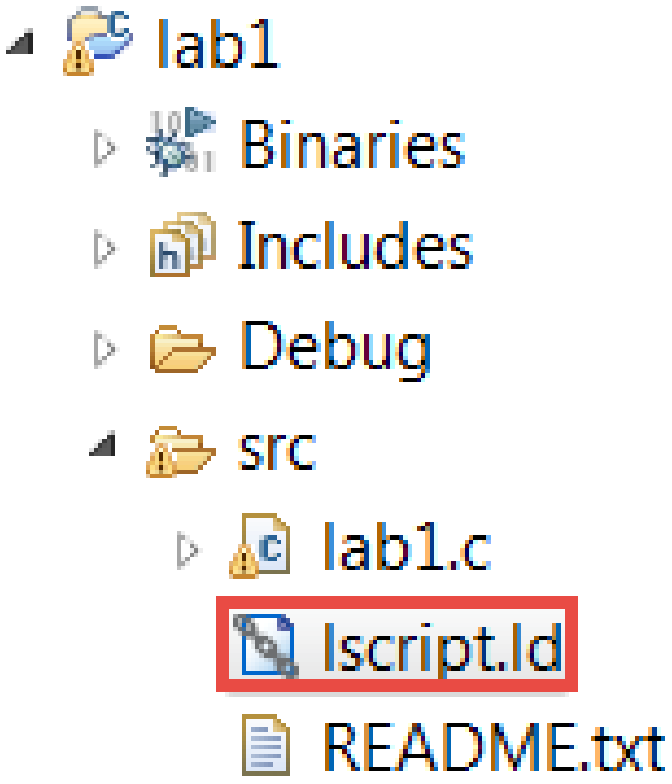
Conclusion

This lab led you through creating the boot images which can boot standalone applications from either the SD card or the QSPI flash memory. You then created the design capable of booting multiple applications and configurations which you developed in the previous labs.

Appendix A-1

Start another instance of the SDK program. Open the lab1 project, change the ps7_ddr_0 to 0x00200000 and the Size to 0x1FE00000 in the lscript.ld (linker script) file. Recompile the lab1.c file. Use objcopy command to convert the elf file into the binary file and note the size of the binary file as well as the program entry point (main()).

1. Start the **SDK** program and browse to the workspace pointing to **{labs}\lab1\lab1.sdk** and click** OK.**
2. Right-click on the **lab1** project, select the *Generate Linker Script* option, change the *code, data, heap, and stack* sections to use the *ps7_ddr_0*, and click **Generate**. Click **Yes** to overwrite the linker script.
3. Expand the **lab1 > src** entry in the Project Explorer, and double-click on the **lscript.ld** to open it.



Accessing the linker script to change the base address and the size

4. In the lscript editor view, change the Base Address of the *ps7_ddr_0_AXI_BASEADDR* from **0x00100000** to **0x00200000** , and the Size from **0x1FF00000** to **0x1FE00000**.

Name	Base Address	Size
ps7_ddr_0	0x200000	0x1FE00000
ps7_qspi_linear_0	0xFC000000	0x1000000
ps7_ram_0	0x0	0x30000
ps7_ram_1	0xFFFF0000	0xFE00

Changing the Base address and the size

5. Press **Ctrl-S** to save the change.

   The program should compile.

6. In the SDK of the **lab1** project, select **Xilinx > Launch Shell** to open the shell session.
7. In the shell window, change the directory to **lab1\Debug** using the **cd** command.
8. Convert the *lab1.elf* file to *lab1elf.bin* file by typing the following command.

   arm-none-eabi-objcopy -O binary lab1.elf lab1elf.bin

9. Type **ls –l** in the shell window and note the size of the file. In this case, it is **32776** , which is equivalent to **0x8008** bytes.
10. Determine the entry point "main()" of the program using the following command in the shell window.

    arm-none-eabi-objdump -S lab1.elf | grep main

    It should be in the **0x002005b8**.

    Make a note of these two numbers (length and entry point) as they will be used in the lab5_sd application.

11. Close the Shell window.

Appendix A-2

Switch the workspace to lab3's SDK project. Assign all sections to ps7_ddr_0 and generate the linker script. Change the ps7_ddr_0 to 0x00600000 and the Size to 0x1FA00000 in the lscript.ld (linker script) file as you did in the previous step. Recompile the lab3.c file. Use the objcopy command to convert the elf file into the binary file and note the size of the binary file as well as the program entry point "main()".

1. In the **SDK** program switch the workspace by selecting **File > Switch Workspace > Other...** and browse to the workspace pointing to **{labs}\lab3\lab3.sdk** and click** OK.**
2. Right-click on the **lab3** entry, select the *Generate Linker Script* option, change the *code, data, heap, and stack* sections to use the *ps7_ddr_0*, and generate the linker script.
3. Change the loop bound from *999999* back to **99999999** in *lab3.c* and save the file.
4. Expand the **lab3 > src** entry in the Project Explorer, and double-click on the **lscript.ld** to open it.
5. In the lscript editor view, change the Base Address of **ps7_ddr_0** from **0x00100000** to **0x00600000** , and the Size from **0x1FF00000** to **0x1FA00000**.
6. Press **Ctrl-S** to save the change.

   The program should compile.

7. In the SDK of the **lab3** project, select **Xilinx > Launch Shell** to open the shell session.
8. In the shell window, change the directory to **lab3\Debug** using the **cd** command.

9. Convert the lab3.elf file to lab3elf.bin file by typing the following command.

   arm-none-eabi-objcopy -O binary lab3.elf lab3elf.bin

10. Type **ls –l** in the shell window and note the size of the file. In this case, it is **32776** again which is equivalent to **0x8008**.

11. Determine the entry point (main()) of the program using the following command in the shell window.

    arm-none-eabi-objdump -S lab3.elf | grep main

    It should be in the **0x006005b8**.

    Make a note of these two numbers (length and entry point) as they will be used in the lab5_sd application.

12. Close the shell window.

13. Exit the SDK of lab3.

Appendix B-1

This step bring in MULTIBOOT register related code in Lab1. It then convert the lab1 executable file to the required (.bin) format.

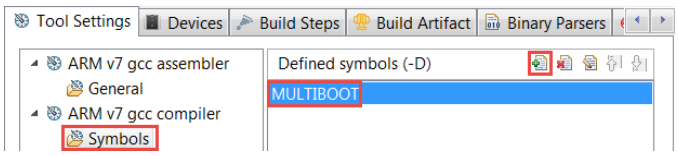
Start a new SDK session and select lab1.sdk as the workspace. Define MULTIBOOT symbol, create Zynq_fsbl application, and change the lab1 BSP reference to zynq_fsbl_bsp. Create the image using the zynq_fsbl.elf, system_wrapper.bit, and lab1.elf files.

1. Start the **SDK** program and browse to the workspace pointing to **{labs}\lab1\lab1.sdk** and click** OK.**

2. Right-click on the **lab1** entry, select the *C/C++ Build Settings* option.

3. Select *Symbols* in the left pane under the *ARM gcc compiler* group, click the + button on the right, enter **MULTIBOOT** in open form, click **OK** and click **OK** again.

   The application will re-compile as the MULTIBOOT related code is now included.



Setting user-defined symbol

4. Select **File** > **New** > **Application Project**

5. Enter **zynq_fsbl** as the project name, select the *Create New* option with **zynq_fsbl_bsp** , and click **Next**.

6. Select *Zynq FSBL* in the **Available Templates** pane and click **Finish**.

7. Select *lab1* in the Project Explorer pane, right-click, and select *Change Referenced BSP*.
8. In the displayed form, select *zynq_fsbl_bsp* and click **OK**.

   The lab1 will be re-compiled using the zynq_fsbl_bsp.
9. In the SDK, select **Xilinx > Create Boot Image**.
10. Select **Create new BIF file** option, click the *Browse* button of the BIF file path field and browse to *{labs}\lab1* directory, set filename as *lab1*, and click **Save**.
11. Add the three files, zynq_fsbl.elf, system_wrapper.bit, and lab1.elf with the correct path and order.
12. Change the output filename to **lab1.bin** making sure that the output directory is *lab1.*
13. Click the **Create Image** button.

    The lab1.bin will be created in the lab1 directory.

Appendix B-2

This step bring in MULTIBOOT register related code in Lab3. It then convert the lab3 executable file to the required (.bin) format.

Switch workspace to lab3.sdk directory. Define MULTIBOOT symbol, create Zynq_fsbl application, and change the lab3 BSP reference to zynq_fsbl_bsp. Create the image using the zynq_fsbl.elf, system_wrapper.bit, and lab3.elf files and naming it as lab3.bin in the lab3 directory.

1. In the **SDK** program switch the workspace by selecting **File > Switch Workspace > Other...** and browse to the workspace pointing to **{labs}\lab3\lab3.sdk** and click** OK.**
2. Right-click on the **lab3** entry, select the *C/C++ Build Settings* option.
3. Select *Symbols* in the left pane under the *ARM gcc compiler* group, click the + button on the right, enter **MULTIBOOT** in open form, click **OK** , and click **OK** again.

   The application will re-compile as the MULTIBOOT related code is now included.
4. Select **File** > **New** > **Application Project**
5. Enter **zynq_fsbl** as the project name, select the *Create New* option with **zynq_fsbl_bsp** , and click **Next**.
6. Select *Zynq FSBL* in the **Available Templates** pane and click **Finish**.
7. Select *lab3* in the Project Explorer pane, right-click, and select *Change Referenced BSP*.
8. In the displayed form, select *zynq_fsbl_bsp* and click **OK**.

   The lab3 will be re-compiled using the zynq_fsbl_bsp.
9. Select the **lab3** application in the *Project Explorer* view.
10. In the SDK, select **Xilinx > Create Boot Image**.
11. Select **Create new BIF file** option, click the *Browse* button of the BIF file path field and browse to *{labs}\lab3* directory, set filename as *lab3*, and click **Save**.
12. Make sure that the three files, **zynq_fsbl.elf, system_wrapper.bit** , and **lab3.elf** file entries are added with the correct path. Change if necessary.

13. Change the output filename to **lab3.bin** making sure that the output directory is lab3.
14. Click the **Create Image** button.

    The lab3.bin will be created in the lab3 directory.
15. Close the SDK session.